Міністерство освіти і науки України
Державний університет „Житомирська політехніка”

Кафедра КТуМтаТ
Група: ТР-25-1

Теорія ймовірностей і математична статистика

Практична робота №7-8

«Формула повної ймовірності та формула Байєса в ймовірнісній класифікації та діагностиці програмних систем»

Перевірив: Лисенко М. С.

Виконав: Наумчук А. В.

					ІКТР.420001.123-ЗЛ		
Змн.	Арк.	№ докум.	Підпис	Дата			
Розроб.		Наумчук А. В.			Звіт з практичної роботи	Літ.	Арк. 1 / Аркушів
Перевір.		Лисенко М. С.					
Керівник						ФІКТ, гр. ТР-25-1	
Н. контр.							
Затверд.							

Мета: засвоїти практичні навички застосування формули повної ймовірності та формули Байєса для аналізу гіпотез, переоцінки ймовірностей під впливом нових даних (апріорні та апостеріорні ймовірності); навчитися будувати прості ймовірнісні класифікатори (Naive Bayes Classifier) для аналізу логів, виявлення аномалій та діагностики дефектів ПЗ мовами програмування високого рівня.

Короткі теоретичні відомості:

1. Повна група гіпотез та формула повної ймовірності

Нехай події H_1, H_2, …, H_n утворюють повну групу несумісних подій (гіпотез), тобто:

$$H_i \cap H_j = \emptyset \ (i \neq j) \text{ та } \Sigma_{i=1}^{n} P(H_i) = 1$$

Якщо подія A може відбутися лише разом із однією з гіпотез H_i, то її ймовірність обчислюється за формулою повної ймовірності

$$P(A) = \Sigma_{i=1}^{n} P(H_i) \cdot P(A|H_i)$$

2. Формула Байєса (Теорема гіпотез)

Якщо подія A вже відбулася в результаті випробування ($P(A) > 0$), то умовна ймовірність того, що при цьому була реалізована саме гіпотеза H_k (апостеріорна ймовірність), обчислюється за формулою Байєса:

$$P(H_k|A) = P(H_k) \cdot P(A|H_k) / P(A) = P(H_k) \cdot P(A|H_k) / \Sigma_{i=1}^{n} P(H_i) \cdot P(A|H_i)$$

-$P(H_k)$ - апріорна ймовірність гіпотези (до проведення експерименту)

-$P(A|H_k)$ - правдоподібність (Likelihood) події A за умови гіпотези H_k

-$P(H_k|A)$ - апостеріорна ймовірність гіпотези (після отримання факту настання події A)

3. Наївний байєсівський класифікатор (Naive Bayes)

Для вектора ознак $X = (x_1, x_2, \ldots, x_m)$ та класу C_k приймається «наївне» припущення про умовну незалежність ознак:

$$P(C_k|X) \propto P(C_k) \cdot \prod_{j=1}^{m} P(x_j|C_k)$$

Прогнозований клас обирається за правилом максимуму апостеріорної ймовірності (MAP):

$$\hat{C} = \text{argmax}_k \left(P(C_k) \cdot \prod_{j=1}^{m} P(x_j|C_k) \right)$$

Виконання роботи:

БЛОК 1.

```
for n in N_values:
    count_A = 0
    count_H3_and_A = 0

    for i in range(n):
        r = random.random()# Визначаємо гіпотезу (хто відправив запит)
        if r < pH1:
            is_A = random.random() < pA_H1
            is_H3 = False
        elif r < pH1 + pH2:
            is_A = random.random() < pA_H2
            is_H3 = False
        else:
            is_A = random.random() < pA_H3
            is_H3 = True

        if is_A:
            count_A += 1
            if is_H3:
                count_H3_and_A += 1

    w_H3_given_A = count_H3_and_A / count_A if count_A > 0 else 0
    frequencies_H3_given_A.append(w_H3_given_A)
    error = abs(w_H3_given_A - pH3_given_A_theor)
```

Рис1.1. – Частина коду яка генерує запити легитимих юзерів, хакерів та ботів.

```
C:\Users\asusp\AppData\Loca
========== БЛОК 1: Байєсівське оновлення (WAF/IDS) ==========
N          | P(H3|A) (стат)  | P(H3|A) (теор)  | Похибка
------------------------------------------------------------
100        | 0.16667         | 0.38298         | 0.21631
1000       | 0.45455         | 0.38298         | 0.07157
10000      | 0.41732         | 0.38298         | 0.03434
100000     | 0.36978         | 0.38298         | 0.01320
1000000    | 0.38004         | 0.38298         | 0.00294
```

Рис1.2 – Вихідні дані у вигляді таблиці.

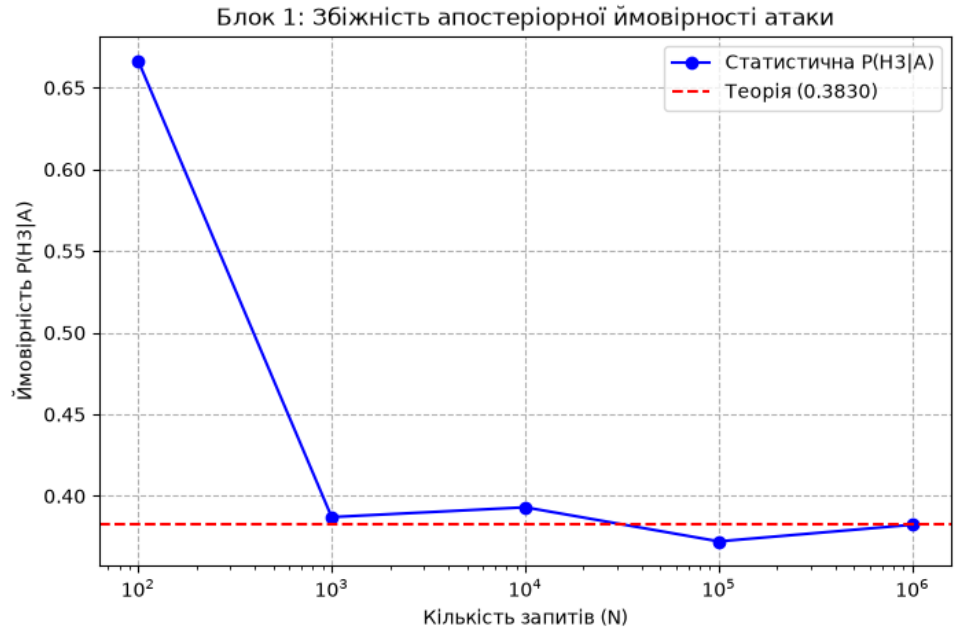


Рис1.3. – Вихідні дані у вигляді графіку.

БЛОК 2. (3)

```python
import random

print("========== Блок 2 (Варіант 3): Джерела веб-трафіку ==========")
N = 100000

pH1, pH2, pH3 = 0.40, 0.40, 0.20# Апріорні ймовірності каналів (H1 - Прямий, H2 - Органіка, H3 - Реклама)
pC_H1, pC_H2, pC_H3 = 0.05, 0.02, 0.10# Правдоподібності покупки C

pC_theor = pH1*pC_H1 + pH2*pC_H2 + pH3*pC_H3
pH3_given_C_theor = (pH3 * pC_H3) / pC_theor

count_C = 0
count_H3_and_C = 0

for i in range(N):
    r = random.random()
    if r < pH1:
        is_C = random.random() < pC_H1
        is_H3 = False
    elif r < pH1 + pH2:
        is_C = random.random() < pC_H2
        is_H3 = False
    else:
        is_C = random.random() < pC_H3
        is_H3 = True

    if is_C:
        count_C += 1
        if is_H3:
            count_H3_and_C += 1
```

Рис2.1 – Частина коду яка виконує симуляцію 100000 візитів із 3-х джерел.

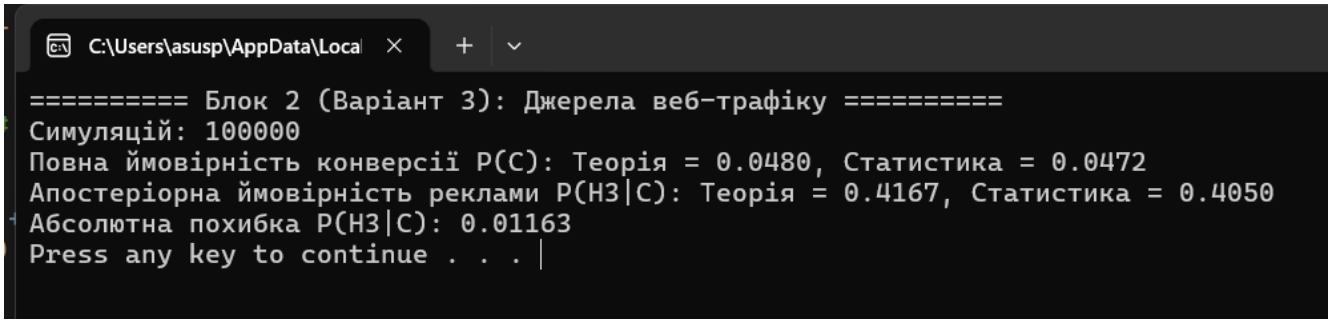


Рис2.2 – Вихідні дані у вигляді інформаційного блоку.

БЛОК 3.

```
12
13  TP, FP, TN, FN = 0, 0, 0, 0 # матриця помилок
14
15  for i in range(N):
16      actual_spam = random.random() < p_Spam# Генерація фактичного класу повідомлення
17
18      if actual_spam:#Генерація ознак (наявність слів w1 та w2)
19          x1 = 1 if random.random() < pw1_Spam else 0
20          x2 = 1 if random.random() < pw2_Spam else 0
21      else:
22          x1 = 1 if random.random() < pw1_Ham else 0
23          x2 = 1 if random.random() < pw2_Ham else 0
24
25      # згладжування не застосовуємо, оскільки ймовірності не нульові
26      prob_X_Spam = (pw1_Spam if x1 else 1 - pw1_Spam) * (pw2_Spam if x2 else 1 - pw2_Spam)
27      score_Spam = p_Spam * prob_X_Spam
28
29      prob_X_Ham = (pw1_Ham if x1 else 1 - pw1_Ham) * (pw2_Ham if x2 else 1 - pw2_Ham)
30      score_Ham = p_Ham * prob_X_Ham
31
32      pred_spam = score_Spam > score_Ham    # прогнозований клас
```

Рис3.1 – Частина коду яка виконує генерацію вибірки випадкових листів.

```
33
34      if actual_spam and pred_spam:
35          TP += 1
36      elif not actual_spam and pred_spam:
37          FP += 1
38      elif not actual_spam and not pred_spam:
39          TN += 1
40      elif actual_spam and not pred_spam:
41          FN += 1
42
43  #обчислення метрик
44  accuracy = (TP + TN) / N
45  precision = TP / (TP + FP) if (TP + FP) > 0 else 0
46  recall = TP / (TP + FN) if (TP + FN) > 0 else 0
47  f1_score = 2 * precision * recall / (precision + recall) if (precision + recall) > 0 else 0
48
```

Рис3.2 – Частина коду яка перевіряє наскільки правильним виявився прогноз

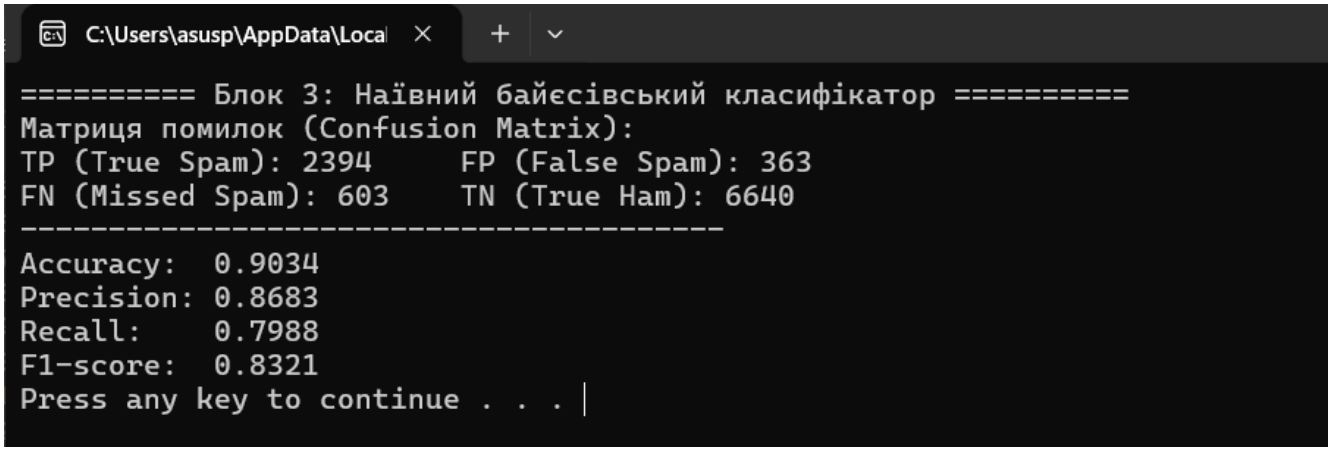


Рис3.3 – Вихідні дані у вигляді інформаційного блоку

БЛОК 4. (8)

```
for n in N_values:
    count_A = 0
    count_H1_and_A = 0

    for _ in range(n):
        is_H1 = random.random() < pH1

        if is_H1:
            is_A = random.random() < pA_H1
        else:
            is_A = random.random() < pA_H2

        if is_A:
            count_A += 1
            if is_H1:
                count_H1_and_A += 1

    w_H1_given_A = count_H1_and_A / count_A if count_A > 0 else 0
    frequencies.append(w_H1_given_A)
    error = abs(w_H1_given_A - pH1_given_A_theor)
```

Рис4.1 – Частина коду яка генерує випадкову подію та підраховує збіги

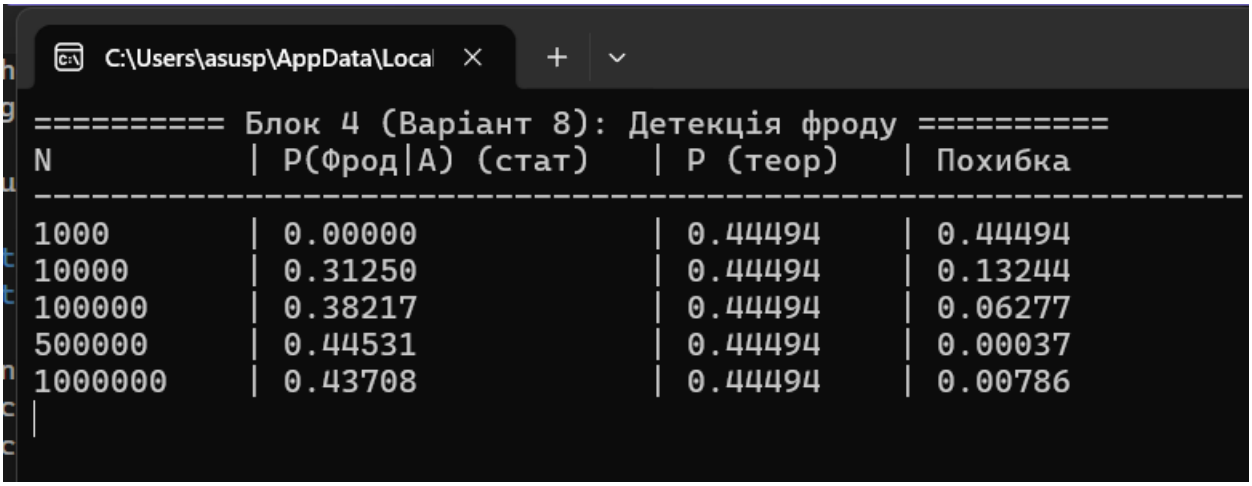


Рис4.2. – Вихідні дані у вигляді таблиці.

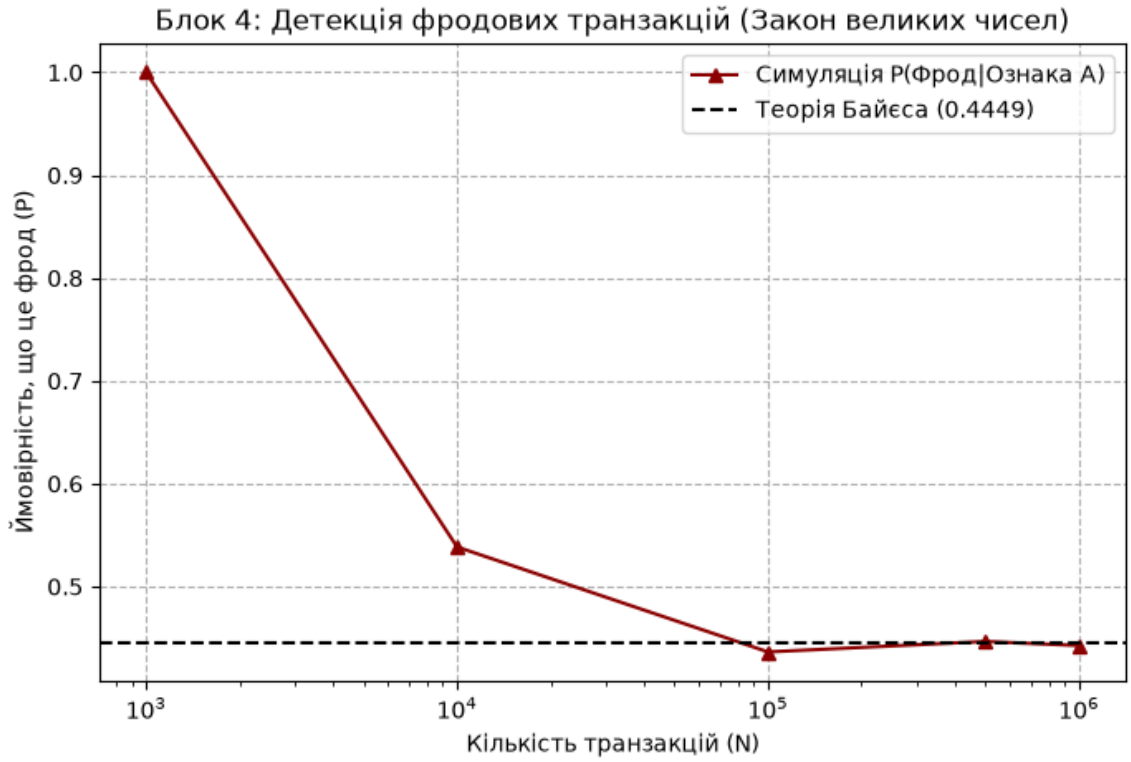


Рис 4.3. – Вихідні дані у вигляді графіку.

Посилання на репрозиторій:

https://github.com/8sepp8/PR7-8_TheoryOfProbaility_Naumchuk_TR-25-1

ВИСНОВОК

Під час виконання практичної роботи я успішно застосував формулу повної ймовірності та теорему Байєса для переоцінки гіпотез на основі нових даних. За допомогою методу Монте-Карло я програмно змоделював роботу аналітичних систем (WAF, класифікація веб-трафіку, детекція фроду), наочно підтвердивши збіжність емпіричних частот із точними теоретичними розрахунками. Також я власноруч реалізував наївний байєсівський класифікатор (Naive Bayes) для розпізнавання спаму та оцінив його ефективність за допомогою матриці помилок і ключових метрик (Accuracy, Precision, Recall, F1-score).